

De la classification des chiffres manuscrits à la détection de cancers du sein

Une introduction aux réseaux de neurones convolutifs (CNN)

Projet de *Mathématiques pour le Machine Learning*

Année 2025-2026 / Semestre de printemps, I1, SM604

Département de Mathématiques de l'EFREI



Objectif du projet

L'objectif de ce projet est de mettre en œuvre les méthodes d'apprentissage supervisé, en particulier les réseaux de neurones convolutifs, pour résoudre des problèmes de classification d'images. Vous verrez que la difficulté évolue significativement lorsque l'on passe de données académiques simples à des images réelles. Vous allez, *in fine*, appréhender des motifs complexes comme des mammographies, ce qui constituera l'aboutissement de ce projet.

Dans un premier temps, vous écrirez un programme permettant de résoudre le problème de classification des chiffres manuscrits (MNIST). Une fois l'algorithme construit, vous en testerez les performances.

Puis, vous coderez la classification d'images en couleur (objets et animaux). Pour cela, vous serez amenés à étudier et implémenter le principe de la convolution.

Enfin, vous vous attaquerez à un jeu de données médicales (mammographies) dans le but de détecter des signes de cancers du sein.

Lors de ce projet, vous coderez dans le langage de votre choix. Attention : l'idée n'est pas d'utiliser uniquement des fonctions "boîtes noires", mais d'implémenter les algorithmes afin d'exercer vos compétences en mathématiques et en informatique.

Votre code aura ses qualités et ses défauts. C'est sa pleine compréhension qui nourrira la discussion lors de votre soutenance orale, que nous souhaitons la plus analytique et critique possible.

1. La base de données MNIST

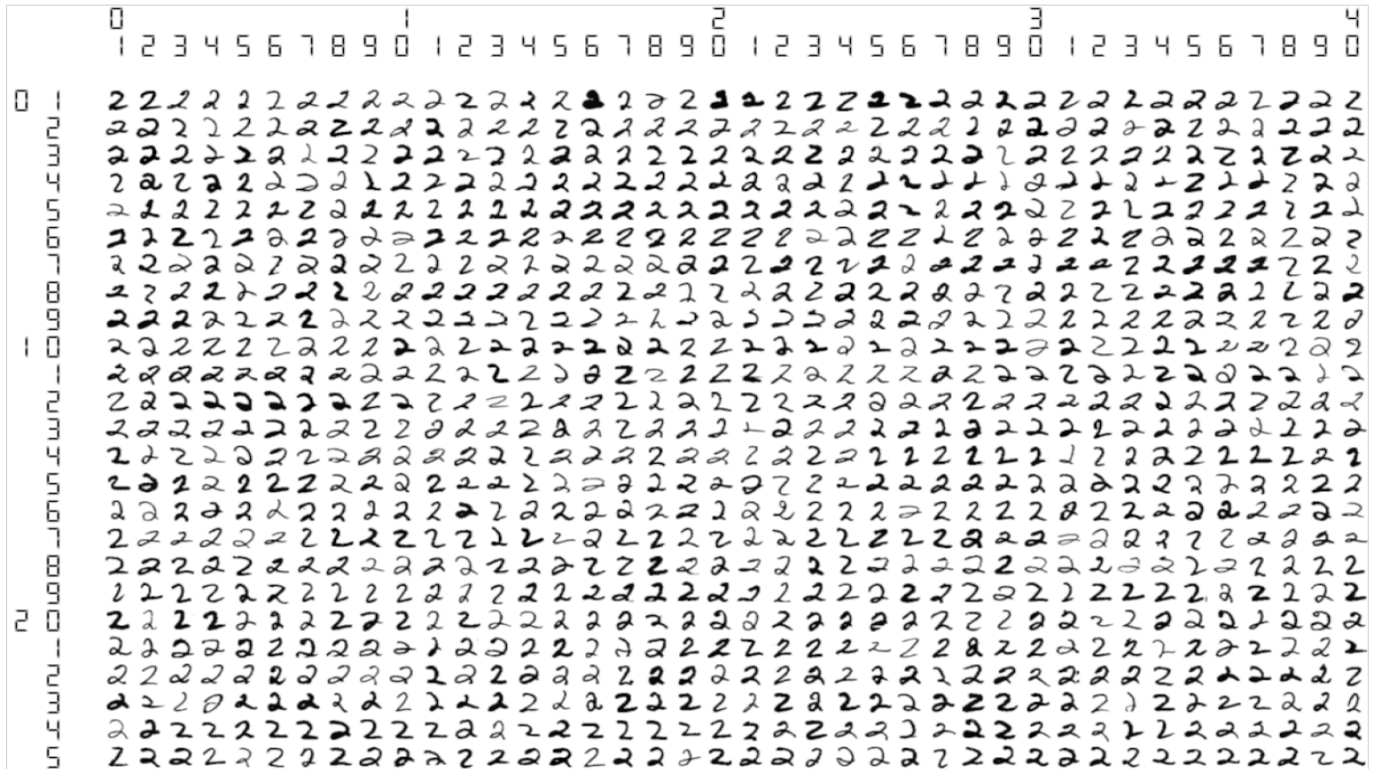
1.1 Le jeu de données

Vous utiliserez la base de données [MNIST](#) : *Modified National Institute of Standards and Technology*. C'est une base de chiffres écrits à la main. Dans la figure 1.1, vous trouverez un exemple de fichier. Dans cette figure, le chiffre "deux" a été écrit 1000 fois à la main dans le format 28×28 pixels.

Nous voulons construire un modèle ajusté tel que pour chaque entrée $\vec{x}$, c'est-à-dire chaque photo, la fonction F , détecte s'il s'agit du chiffre "deux" écrit à la main. Bien évidemment, nous voulons aussi détecter les autres chiffres écrits à la main. C'est donc un problème de classification multi-classe.

Ce problème correspond à considérer un ensemble de n données et d'étiquettes $\{(\vec{x}_i, y_i)\}_{i=1}^n$ où chaque image i est représentée par un vecteur $\vec{x}_i \in \mathbb{R}^{784}$ dont les coordonnées correspondent aux valeurs d'intensité de chaque pixel (on ne travaillera pas en binaire), et où $y_i \in \{0, \dots, 9\}$ désigne la classe associée. L'objectif est de construire une fonction F paramétrée par un ensemble de paramètres stockés dans le vecteurs A , afin de prédire la classe d'une image.

Mais avant d'implémenter un modèle simple de classification et d'analyser son comportement, vous allez être amené à importer les images des chiffres manuscrits et les représenter sous forme de vecteurs $\vec{x}_i \in \mathbb{R}^{784}$. Il faudra ensuite normaliser les données si nécessaire. Et séparer les données en un ensemble d'entraînement et un ensemble de test.



IMAGES/GROUPS/mnist_v5_MNIST-2_01001-02000_25x40.png

FIGURE 1.1 – 1000 chiffres "deux", et chaque chiffre manuscrit est en 28×28 pixels.

1.2 Le modèle de classification

Une fois ce travail préalable de mise en forme des données effectué, vous devrez :

- tester un modèle de classification en utilisant le chapitre 3 de votre cours.
- puis tester un modèle à plusieurs couches en utilisant le chapitre 4 de votre cours.
- discuter de vos résultats.

1.2.1 Modèle linéaire

Vous implémenterez d'abord un modèle F multi-classe linéaire, sans couche cachée, afin de construire des scores. Ainsi, vous considérerez le modèle linéaire multi-classe qui, pour chaque entrée $\vec{x} = (x_j)_{j \in \llbracket 1, 784 \rrbracket}$, calcule un score (ou *logit*) défini par :

$$o_k = \sum_{j=1}^{784} a_{k,j} x_j + a_{k,0}, \text{ avec } k \in \{0, \dots, 9\}$$

La matrice A des paramètres à apprendre est telle que $A = (a_{k,j})_{k \in \llbracket 0,9 \rrbracket, j \in \llbracket 1,784 \rrbracket}$, le vecteur des scores est $o = (o_k)_{k \in \llbracket 0,9 \rrbracket} \in \mathbb{R}^{10}$ et $b = (a_{k,0})_{k \in \llbracket 0,9 \rrbracket}$ regroupe les biais.

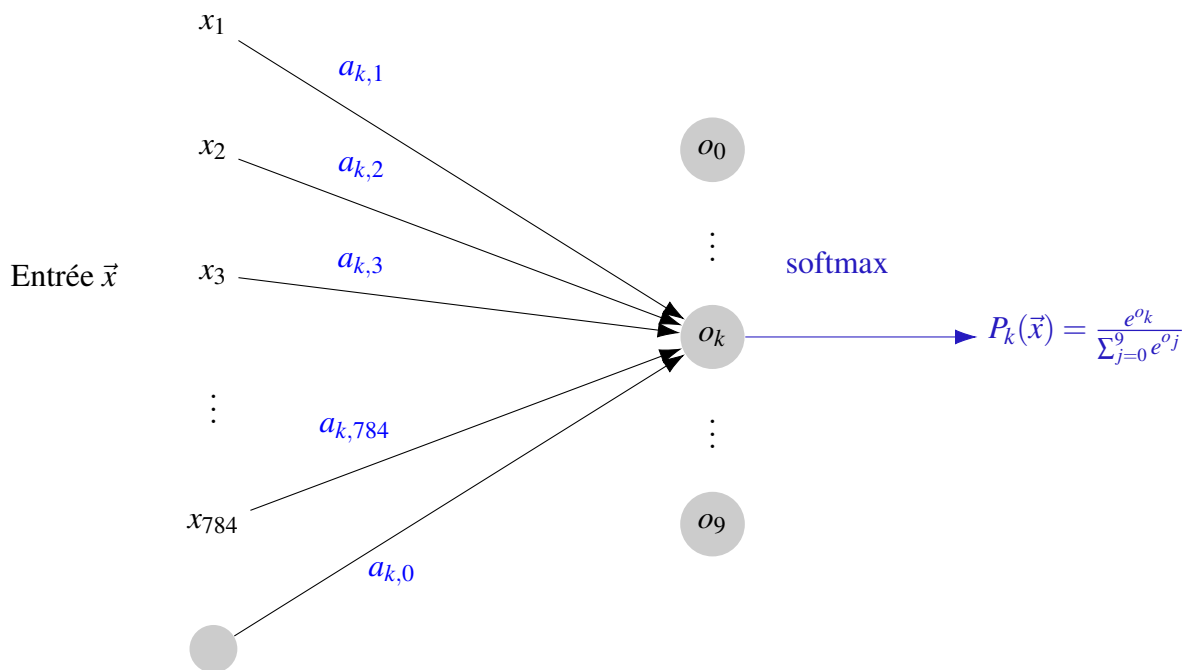
On peut écrire, pour chaque entrée : $\vec{x}$:

$$o = A\vec{x} + b$$

Attention : o n'est pas le vecteur nul, ni zéro, mais le vecteur des scores.

En résumé :

- $\vec{x}$ est un vecteur colonne de taille 784×1 .
- A est une matrice de taille 10×784 . Chaque ligne k contient tous les poids pour la classe k .
- b est un vecteur de taille 10×1 contenant les biais $a_{k,0}$.
- o est le résultat : un vecteur de 10 scores.



Ces scores sont transformés en probabilités à l'aide de la fonction *softmax* : $P_k(\vec{x}) = \frac{e^{o_k}}{\sum_{j=0}^9 e^{o_j}}$

La prédiction du modèle est faite avec : $\hat{y} = \operatorname{argmax}_k (P_k(\vec{x}))$

Pour l'apprentissage, vous utiliserez la fonction de coût entropie croisée ou cross-entropy dans un modèle multi-classe :

Définition (L'entropie croisée ou cross-entropy).

Soit un modèle de classification multi-classe qui, pour chaque entrée $\vec{x}$, prédit un vecteur de probabilités $P = (P_0, \dots, P_9)$ où $P_k = \mathbb{P}(Y = k \mid \vec{x})$.

Soit l'ensemble des vraies étiquettes $y = (y_i)_{i \in \llbracket 1, n \rrbracket} \in \{0, \dots, 9\}^n$ associées aux données. On définit une nouvelle matrice des étiquettes :

$$(y_i^{(k)})_{i \in \llbracket 1, n \rrbracket, k \in \llbracket 0, 9 \rrbracket} \text{ telle que } \begin{cases} y_i^{(k)} = 1 & \text{si l'image } i \text{ appartient à la classe } k \\ y_i^{(k')} = 0 & \text{pour } k' \neq k \end{cases}$$

On définit la **fonction coût Log Loss** par :

$$\mathcal{L}(y, P) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=0}^9 \mathcal{L}_{i,k} = -\frac{1}{n} \sum_{i=1}^n \sum_{k=0}^9 y_i^{(k)} \cdot \ln(P_k(\vec{x}_i))$$

Avant de commencer, vous calculerez la formule du gradient. Puis, vous effectuerez une descente de gradient sur ce jeu de données, jusqu'à trouver des paramètres satisfaisants. Vous calculerez alors le taux d'erreur de votre modèle.

1.2.2 Modèle avec couches cachées

Vous implémenterez ensuite un modèle F sous la forme d'un réseau de neurones multi-couches avec H couches cachées, comme dans le chapitre 4 du cours :

- La sortie z_q^1 du q -ème neurone de la première couche cachée vaut, avec ϕ_1 la fonction d'activation de la première couche cachée :

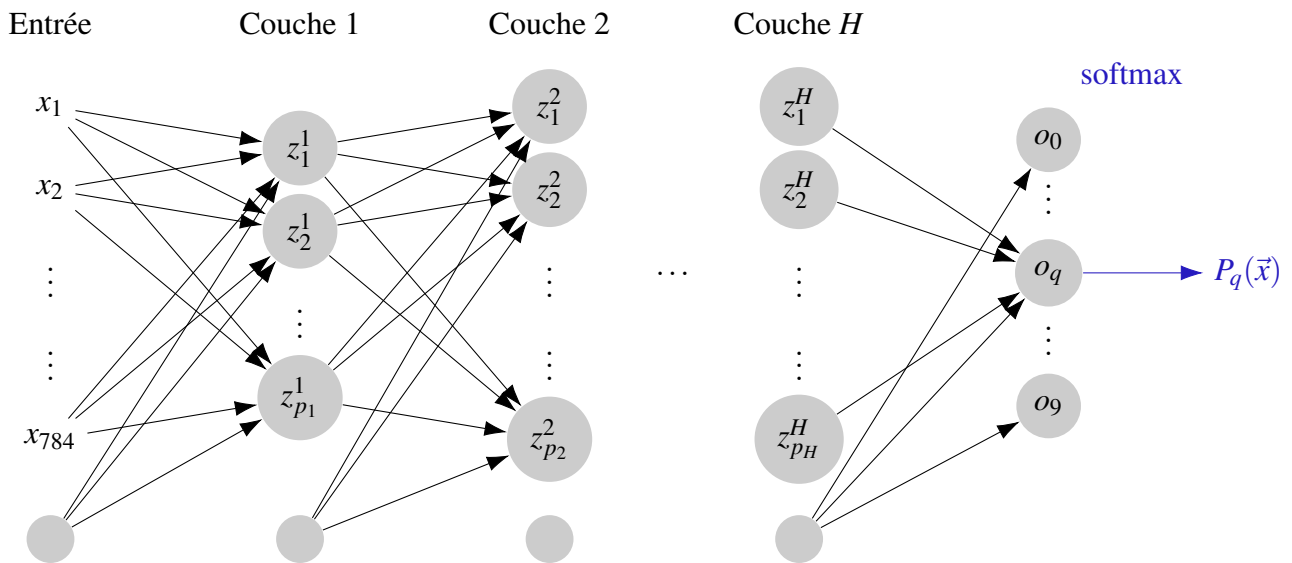
$$z_q^1 = \phi_1 \left(\sum_{k=1}^{784} a_{qk}^1 x_k + a_q^1 \right) = \phi_1(o_q^1)$$

- La sortie z_q^2 du q -ème neurone de la deuxième couche cachée vaut, avec ϕ_2 la fonction d'activation de la deuxième couche cachée :

$$z_q^2 = \phi_2 \left(\sum_{k=1}^{p_1} a_{qk}^2 z_k^1 + a_q^2 \right) = \phi_2(o_q^2)$$

- Le score o_q du q -ème neurone de la dernière couche vaut :

$$o_q = \sum_{k=1}^{p_H} a_{qk}^{H+1} z_k^H + a_q^{H+1} \text{ pour } q \in \{0, \dots, 9\}$$



On utilise l'activation *softmax* à la sortie du perceptron sur $o = (o_0, \dots, o_9)$. Puis la prédiction du modèle se fait avec : $\hat{y} = \operatorname{argmax}_k (\operatorname{softmax}(o))$

Attention : dans la notation z_q^h , le h n'est pas une puissance mais l'indice de la couche. De même pour a_{qk}^h .

Maintenant que l'architecture est définie, vous coderez la descente de gradient avec une, puis deux couches cachées (i.e. $H = 1$ puis $H = 2$). Vous comparerez ensuite les résultats de la prédiction avec le modèle linéaire, en évaluant les taux d'erreur.

Pour cela, vous préciserez :

- la dimension des variables z^1 , c'est à dire la valeur de p_1 . Idem pour la deuxième couche (quand elle apparaît).
- la fonction d'activation choisie pour la ou les couches cachées.
- la fonction de coût.

Vous implémenterez bien entendu la rétro-propagation du chapitre 5.

1.2.3 L'évaluation, l'analyse des erreurs et la discussion

Afin de comparer le modèle linéaire et le modèle avec une couche, vous devrez :

- calculer le taux d'erreur sur l'ensemble d'entraînement
- calculer le taux d'erreur sur l'ensemble de test

Vous étudierez ensuite les chiffres mal classés et tenterez d'expliquer les erreurs observées. On s'intéressera en particulier :

- aux chiffres ambigus
- aux écritures atypiques
- aux limitations du modèle utilisé

Vous chercherez notamment une représentation en deux dimensions pour illustrer votre analyse.

Enfin, vous discuterez :

- des limites du modèle
- du rôle du nombre de paramètres
- des raisons pour lesquelles ce problème reste plus simple que la classification d'images naturelles

2. La base de données CIFAR-10

Nous allons à présent nous servir des mêmes architectures bâties précédemment, afin de classifier des images réelles en couleur.

Nous allons travailler avec le jeu de données [CIFAR-10](#) qui est une base de données de référence en vision par ordinateur. Elle a été créée par l'Institut Canadien de Recherches Avancées (acronyme : CIFAR)

2.1 Le jeu de données

Les images sont en couleurs, mais en basse résolution. La bse contient 60 000 images en couleur de taille 32×32 pixels réparties en :

- 50 000 images d'entraînement
- 10 000 images de test.

Chaque image est ainsi représentée par une matrice de taille $32 \times 32 \times 3$, où la troisième variable correspond aux composantes Rouge, Vert et Bleu (RGB).

Les classes des étiquettes sont au nombre de 10 :

{avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, bateau, camion}



FIGURE 2.1 – 100 images aléatoires de cette banque de données, classées en 10 classes.

Contrairement à MNIST (images en niveaux de gris 28×28), CIFAR-10 présente des images en couleur, des objets plus complexes, un bruit de fond important, une variabilité de position et d'orientation.

2.2 Travail préliminaire

Dans un premier travail préliminaire, vous testerez les précédentes architectures que vous avez construites pour le jeu de données MNIST, en les adaptant. Et vous donnerez les taux d'erreur.

Pour cela vous ferez deux études :

- d'abord, vous convertirez toutes les images $\vec{x}$ (d'entraînement et de test) en niveaux de gris par une combinaison linéaire des intensités des trois canaux.

Pour une image $\vec{x} = (x_j)_{j \in \llbracket 1, 1024 \rrbracket}$ (car $32 \times 32 = 1024$) :

$$x_j = 0.299R_j + 0.587G_j + 0.114B_j$$

R_j étant l'intensité de la composante rouge du pixel j . Les trois coefficients de l'expression sont standards pour obtenir une image en niveaux de gris.

Puis, vous calculerez les taux d'erreur pour le modèle linéaire et le modèle à couches.

- dans un deuxième temps, vous travaillerez avec les trois composantes de couleur. Vous testerez les images qui sont chacune un vecteur $\vec{x} \in \mathbb{R}^{3072}$, 3072 étant égal à $32 \times 32 \times 3$. Vous calculerez alors les taux d'erreur pour le modèle linéaire et le modèle à couches

Remarque :

Vous comparerez les différents taux d'erreur avec ceux des articles scientifiques suivants :

Titre original de l'article	taux d'erreur (%)	Date de publication
Convolutional Deep Belief Networks on CIFAR-10	21,1	août 2010
Maxout Networks	9,38	février 2013
Fractional Max-Pooling	3,47	décembre 2014
Densely Connected Convolutional Networks	3,46	août 2016
Coupled Ensembles of Neural Networks	2,68	septembre 2017
An Image is Worth 16x16 Words : Transformers for Image Recognition at Scale	0,5	juin 2021

Attention : ces performances sont obtenues avec des architectures profondes et des techniques avancées d'optimisation. Les résultats que vous obtiendrez seront naturellement plus modestes.

2.3 Les réseaux neuronaux convolutifs

Jusqu'alors, nous avons directement présenté à notre architecture les entrées $\vec{x}$. A présent, afin de baisser les taux d'erreur précédemment calculés, nous allons introduire les **réseaux de neurones convolutionnels**. L'idée est de modifier cette entrée pour en extraire des informations plus "visibles" par l'architecture de neurones qui va suivre. Pour cela, on va passer la photo à travers des filtres de convolution.

2.3.1 Le principe sur une photo en noir et blanc

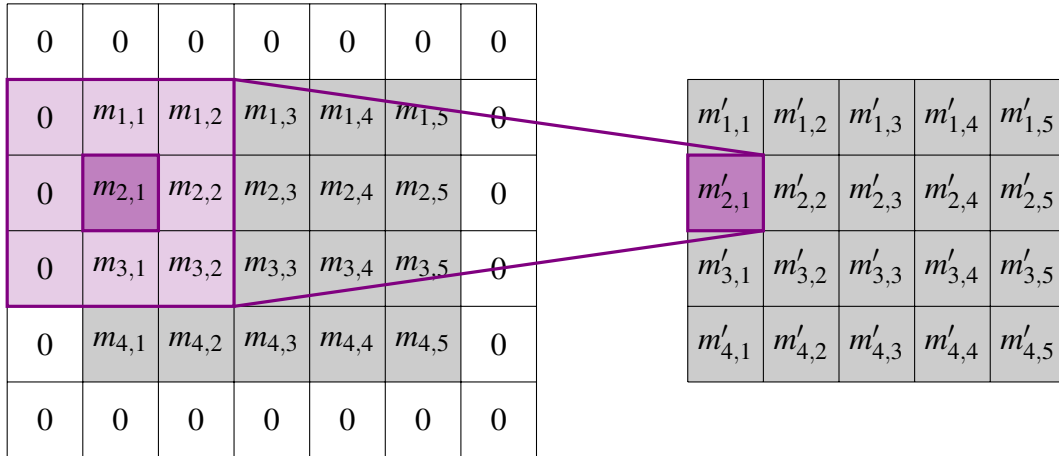
Nous allons effectuer une convolution. Cela consiste à faire glisser un cadre K en tout bloc de pixels sur la photo $\vec{x}$. Cela produit une nouvelle image appelée image filtrée. Voici le cadre :

$$K = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & k \end{bmatrix}$$

On considère les pixels de la photo écrits sous la forme de leur coordonnées : $M = (m_{u,v})_{(u,v) \in \llbracket 1,32 \rrbracket \times \llbracket 1,32 \rrbracket}$ avec (u,v) les coordonnées de chaque pixel et $m_{u,v}$ l'intensité de ce pixel.

Dans un premier temps, afin de conserver la taille de l'image, on ajoute une bordure de zéros autour de l'image (zero-padding). Puis on effectue une convolution qui consiste à superposer le filtre K sur le pixel en (u,v) .

Pour $(u = 2, v = 1)$, cela fait :



On obtient alors une nouvelle photo dont la valeur de l'intensité de chaque pixel (u,v) est $(m'_{u,v})_{(u,v) \in \llbracket 1,32 \rrbracket \times \llbracket 1,32 \rrbracket}$ se déduit par le calcul de :

$$m'_{u,v} = a.m_{u-1,v-1} + b.m_{u-1,v} + c.m_{u-1,v+1} + d.m_{u,v-1} + e.m_{u,v} + f.m_{u,v+1} + g.m_{u+1,v-1} + h.m_{u+1,v} + k.m_{u+1,v+1} + l$$

Remarque : Dans la photo initiale, on comprend l'utilité de la présence des zéros autour de la photo : ils permettent de calculer les valeurs sur les bords de la photo, comme ici :

$$\begin{aligned} m'_{2,1} &= a.m_{1,0} + b.m_{1,1} + c.m_{1,2} + d.m_{2,0} + e.m_{2,1} + f.m_{2,2} + g.m_{3,0} + h.m_{3,1} + k.m_{3,2} + l \\ &= a.0 + b.m_{1,1} + c.m_{1,2} + d.0 + e.m_{2,1} + f.m_{2,2} + g.0 + h.m_{3,1} + k.m_{3,2} + l \end{aligned}$$

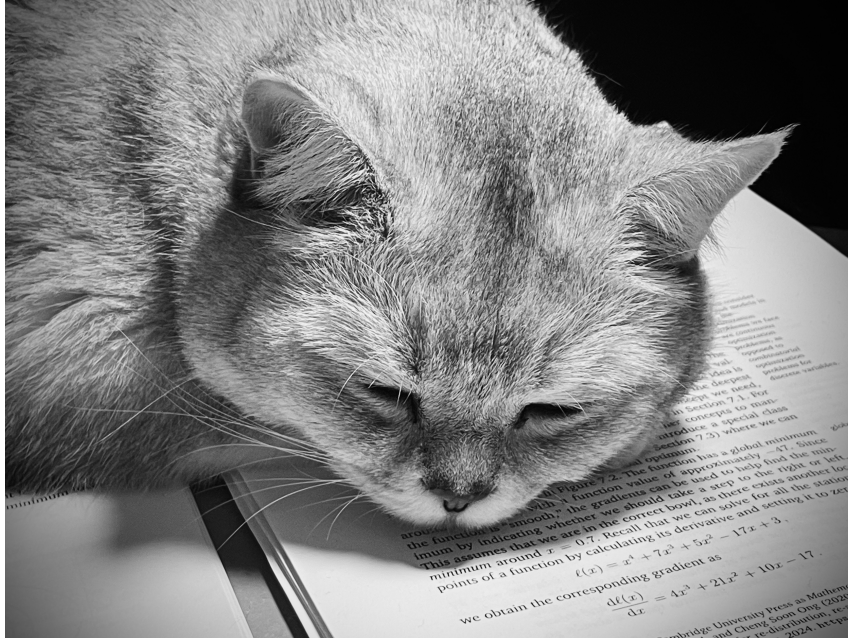
De plus, au lieu d'écrire $m'_{u,v}$ avec $(a, b, c, d, e, f, g, h, k, l)$, on pourrait écrire les 9 constantes dans une matrice K de taille 3×3 , puis on ajoute le biais l et on fait le calcul de la manière suivante :

$$m'_{u,v} = \sum_{u'=1}^3 \sum_{v'=1}^3 K_{u',v'} m_{u'+u-2, v'+v-2} + l$$

On génère ainsi, ce qu'on appelle en Deep Learning, la carte de caractéristiques (**feature map**). Ainsi, l'image filtrée par K peut représenter des bords, des textures, etc.

2.3.2 Le premier travail à faire

Avant d'implémenter tout cela, vous allez travailler d'abord sur la photo suivante :



Vous étudierez l'effet visuel, sur cette photo des filtres, K suivants, pour $l = 0$:

$$K_1 = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}, \quad K_2 = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}, \quad K_3 = \begin{pmatrix} -1 & 2 & -1 \\ -1 & 2 & -1 \\ -1 & 2 & -1 \end{pmatrix}$$

$$K_4 = \begin{pmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{pmatrix}, \quad K_5 = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \quad K_6 = \begin{pmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{pmatrix}$$

2.3.3 Le principe sur une photo en couleur

Sur une photo en couleur, le principe est le même, on peut appliquer des cadres $K^{(R)}$, $K^{(G)}$ et $K^{(B)}$ (les mêmes ou différents) sur les composantes R, G ou B. On forme alors une nouvelle photo dont la valeur de l'intensité de chaque pixel (u, v) est $(m'_{u,v})_{(u,v) \in \llbracket 1,32 \rrbracket \times \llbracket 1,32 \rrbracket}$ se déduit par le calcul de :

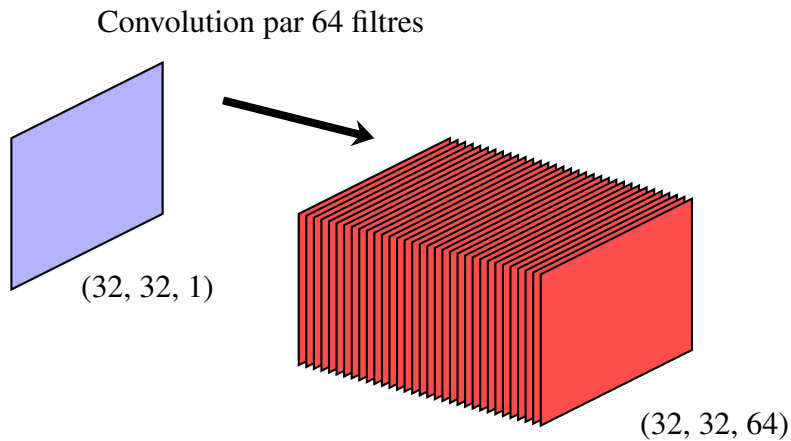
$$m'_{u,v} = \sum_{u'=1}^3 \sum_{v'=1}^3 K_{u',v'}^{(R)} m_{u+u'-2, v+v'-2}^{(R)} + K_{u',v'}^{(G)} m_{u+u'-2, v+v'-2}^{(G)} + K_{u',v'}^{(B)} m_{u+u'-2, v+v'-2}^{(B)} + l$$

avec l le biais.

2.4 Les couches de convolution

En pratique, un seul filtre K ne suffit pas à capturer la complexité d'une image. On applique donc souvent plusieurs filtres $K_1, \dots, K_C$ en parallèle pour générer plusieurs cartes de caractéristiques. Il en existe de plusieurs types. On va donc chercher à capturer la complexité d'une image par plusieurs filtres.

Voici un exemple avec une photo de taille 32×32 en entrée. Elle est d'épaisseur 1, donc on écrit $32 \times 32 \times 1$. Puis on applique 64 filtres de convolution. Ainsi, on obtient en sortie 64 images 32×32 , donc un ensemble d'épaisseur 64, c'est pour cela que l'on écrit $32 \times 32 \times 64$.



On va maintenant définir le Max-Pooling et les filtres en 3D afin de travailler sur cet ensemble qui a une épaisseur.

2.4.1 Le Max-Pooling

Après avoir appliqué une convolution, on obtient une carte de caractéristiques (feature map). Afin de réduire la dimension spatiale de ces données et de ne conserver que les informations les plus pertinentes, on utilise généralement une couche de **Max-Pooling** (ou sous-échantillonnage par le maximum).

Cela consiste à parcourir la carte de caractéristiques M' avec une fenêtre glissante (généralement de taille 2×2) et à ne conserver que la valeur maximale au sein de cette fenêtre. En utilisant un pas de déplacement (**stride**) égal à la taille de la fenêtre, on divise par deux la largeur et la hauteur de l'image.

Si l'on note $M'' = (m''_{u,v})$ la carte de caractéristiques après pooling 2×2 , la valeur du pixel (u, v) en sortie se calcule par :

$$m''_{u,v} = \max_{(u',v') \in \{0,1\}^2} m'_{2u+u'-1, 2v+v'-1}$$

Cette opération présente deux avantages majeurs pour notre réseau :

- **Réduction de dimension** : Pour une image 32×32 en entrée, le Max-Pooling produit une image 16×16 . Cela réduit le nombre de paramètres et le temps de calcul pour les couches suivantes.
- **Invariance aux petites translations** : Si un motif (un contour par exemple) se déplace d'un seul pixel dans l'image d'origine, sa valeur maximale dans la fenêtre de pooling restera probablement inchangée, rendant le modèle robuste.

2.4.2 La convolution sur des volumes : le principe des filtres 3D

Dans un réseau de neurones convolutif, les données ne restent pas sous la forme d'images à 3 canaux (RGB). En effet, on a vu qu'une convolution à C filtres génère un objet mathématique qui possède une "épaisseur" ou une "profondeur" égale à C . Dans l'exemple donné, on avait pris $C = 64$.

On va maintenant traiter ce volume par un nouveau type de filtre de convolution qui va être à trois dimensions : sa taille est $3 \times 3 \times C$. L'opération de convolution consiste à sommer les produits pixel à pixel sur toute l'épaisseur du volume, comme avant avec le filtre violet K , mais cette fois-ci avec un filtre à trois dimensions (un parallélépipède de 3 pixels de large, 3 pixels de long et C pixels de profondeur) dans l'ensemble $32 \times 32 \times C$:

$$m'_{u,v} = \sum_{c=1}^C \left(\sum_{u'=1}^3 \sum_{v'=1}^3 K_{u',v',c} \cdot m_{u+u'-2, v+v'-2, c} \right) + l$$

Ce qu'il faut retenir : **Un filtre 3D produit une seule** carte de caractéristiques qui devient à deux dimensions. On "écrase" en quelque sorte l'épaisseur par la sommation.

2.4.3 L'étape finale : l'aplatissement

Une fois que l'image est passée par plusieurs couches de convolution et de pooling, on obtient un volume de petite taille spatiale mais généralement de grande profondeur (par exemple $4 \times 4 \times 64$).

Afin de pouvoir utiliser les couches linéaires dont on a bâti l'architecture précédemment, on "aplatit" ce volume en un unique vecteur colonne $\vec{x}_{final}$. C'est l'étape de l'**aplatissement** ou **flattening**. Ce vecteur contient toutes les caractéristiques extraites par les convolutions et est ensuite envoyé vers la couche de scores (Logits) et la fonction Softmax.

2.5 Architecture convolutive envisagée

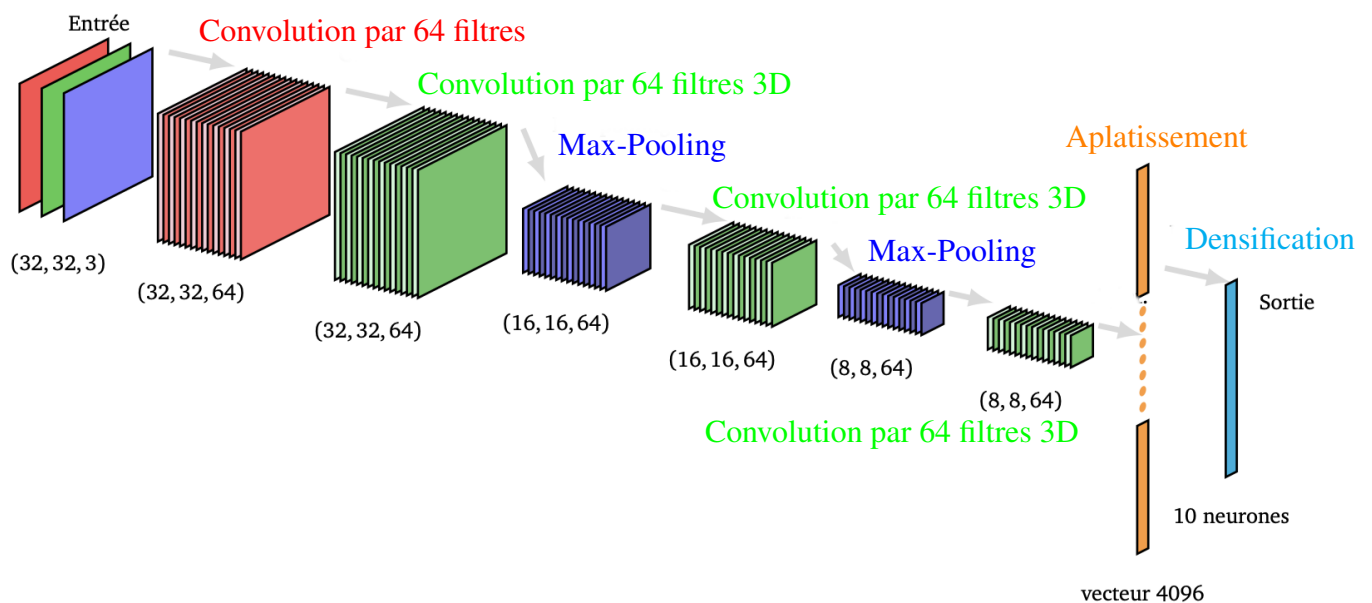
Maintenant que toutes les transformations sur l'image en entrée sont définies, on va définir le réseau convolutif.

2.5.1 L'architecture

On considère une image couleur $\vec{x} \in \mathbb{R}^{32 \times 32 \times 3}$ du jeu de données CIFAR-10 :

- 32×32 représente la résolution spatiale de l'image,
- 3 correspond aux trois canaux de couleur : Rouge, Vert et Bleu.

L'architecture globale est la suivante :



2.5.2 Première couche : convolution par 64 filtres

On applique 64 filtres, comme décrits en 2.3.3, de convolution en couleur. La première couche comporte donc 64 filtres couleur. Ainsi, après l'application des 64 filtres, on obtient une sortie de taille $32 \times 32 \times 64$, c'est-à-dire 64 cartes de caractéristiques.

2.5.3 Deuxième couche : convolution par 64 filtres 3D

Sur ces 64 cartes, on applique 64 filtres 3D différents. Chaque filtre est un parallélépipède de taille $3 \times 3 \times 64$, donc la sortie est de $32 \times 32 \times 64$.

Chaque nouveau filtre combine donc les informations provenant des 64 cartes de caractéristiques produites par la première convolution.

2.5.4 Troisième couche : le Max-Pooling

On applique une opération de *max pooling* de taille 2×2 . Cette opération consiste à remplacer chaque bloc 2×2 par la valeur maximale de ce bloc. La résolution spatiale est donc divisée par deux :

$$32 \times 32 \times 64 \longrightarrow 16 \times 16 \times 64$$

Le nombre de cartes de caractéristiques reste inchangé.

2.5.5 Les autres couches

Puis, on applique une nouvelle convolution par 64 filtres 3D, suivie d'un Max-Pooling, puis d'une dernière convolution par 64 filtres 3D.

On obtient alors un objet de taille $8 \times 8 \times 64 = 4096$ réels que l'on aligne lors de l'aplatissement.

On relie les 4096 réels avec les 10 neurones de sortie, puis on effectue un softmax.

2.6 Apprentissage par rétropropagation

L'objectif de cette section est de mettre à jour les poids de tous nos filtres et les paramètres de nos couches afin de minimiser la fonction de coût.

2.6.1 Option A (difficile) : Le défi mathématique (Calcul et programmation explicite)

Pour les étudiants souhaitant comprendre les rouages internes de l'apprentissage, vous calculerez les gradients manuellement en utilisant la **règle de la chaîne** (chain rule).

Soit $\mathcal{L}$ la fonction de coût. Vous devrez déterminer pour chaque couche h :

1. **Le gradient par rapport aux sorties** : $\frac{\partial \mathcal{L}}{\partial o_q^h}$.
2. **Le gradient par rapport aux paramètres** : $\frac{\partial \mathcal{L}}{\partial K_{u',v',c}^h}$ pour les filtres et $\frac{\partial \mathcal{L}}{\partial a_{qk}^h}$ pour les couches.

Pour la convolution : Le gradient de la perte par rapport à un poids du filtre K est lui-même une opération de convolution. Si l'on note $\delta_{u,v} = \frac{\partial \mathcal{L}}{\partial m_{u,v}}$, alors :

$$\frac{\partial \mathcal{L}}{\partial K_{u',v',c}} = \sum_{u,v} \delta_{u,v} \cdot m_{u+u'-2, v+v'-2, c}$$

Attention : Pour le Max-Pooling, le gradient ne se propage que vers le pixel qui a été sélectionné comme maximum lors de la passe avant (forward). Les trois autres pixels du bloc reçoivent un gradient nul.

2.6.2 Option B : Utilisation d'un framework (PyTorch)

Si le calcul explicite des gradients des filtres 3D vous semble trop complexe, vous pouvez utiliser la bibliothèque **PyTorch**. PyTorch utilise le *graphe de calcul dynamique* pour effectuer automatiquement la rétropropagation (Autograd).

Dans ce cas, votre travail consistera à :

- Définir l'architecture en utilisant les modules `torch.nn.Conv2d`, `torch.nn.MaxPool2d` et `torch.nn.Linear`.
- Implémenter la méthode `forward` qui décrit le passage des données à travers les couches.
- Utiliser un optimiseur (comme `optim.SGD` ou `optim.Adam`) pour mettre à jour les poids après chaque appel à `loss.backward()`.

Couche théorique	Équivalent PyTorch
Convolution 3D	<code>nn.Conv2d(in_channels, out_channels, kernel_size=3)</code>
Max-Pooling	<code>nn.MaxPool2d(kernel_size=2, stride=2)</code>
Aplatissement	<code>torch.flatten(x, 1)</code>
Densification	<code>nn.Linear(in_features, out_features)</code>

2.6.3 Le travail à faire

Vous devez entraîner le réseau convolutionnel afin de trouver les meilleurs paramètres. Puis vous comparerez les performances avec les précédentes architectures.

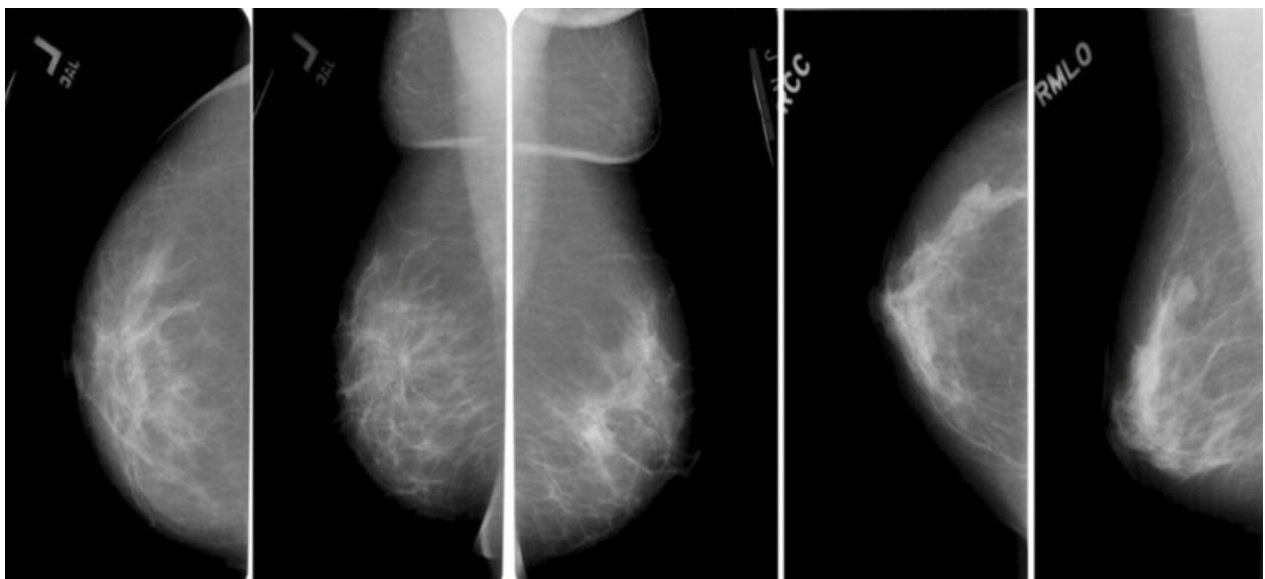
Une attention particulière devra être portée à :

- l'architecture choisie
- le nombre de paramètres
- le phénomène d'overfitting

3. Application au diagnostic médical

Nous arrivons à l'aboutissement du projet. Vous allez maintenant travailler sur la base de données de référence **CBIS-DDSM** (Curated Breast Imaging Subset of DDSM). Chaque image est plus complexe que MNIST ou CIFAR-10, avec une variabilité importante de texture et d'apparence clinique.

Contrairement aux datasets précédents, les étiquettes ne sont pas directement contenues dans le nom des dossiers. Vous devrez les extraire à partir du fichier `mass_case_description_train_set.csv`. La colonne cible est **pathology** : vous effectuerez une classification binaire en regroupant les catégories *BENIGN* et *BENIGN_WITHOUT_CALLBACK* par opposition aux cas *MALIGNANT*.



Vous allez effectuer une classification binaire simple : bénin vs malin (cancer), en utilisant les architectures précédemment étudiées.

Vous devrez :

- **Prétraitement** : Redimensionner les images (très haute résolution à l'origine) et normaliser les niveaux de gris.
- **Matching** : Lier chaque image à son étiquette grâce au chemin de fichier indiqué dans le CSV.
- **Entraînement** : Adapter vos architectures convolutives à ce problème binaire.
- **Analyse** : Étudier la matrice de confusion et discuter de l'importance des faux négatifs en diagnostic médical.

Attention :

Le passage à ce dataset impose deux nouveaux défis :

- Le déséquilibre des classes : il y a une différence de nombre de cas bénins et malins.
- Le redimensionnement : les images d'origine font parfois 4000×3000 pixels. Un redimensionnement vers 128×128 ou 224×224 est indispensable afin d'éviter des problèmes de mémoire (RAM).

4. Le travail à faire

Le planning

- Semaine 13 : création des 8 équipes de votre classe
- Semaine 14 : distribution du sujet de projet
- Semaine 18 : dépôt de votre première partie de projet
- Semaines 19 et 20 : soutenance de votre première partie de projet.
- Semaine 24 : dépôt de votre projet final, après retravail, sur Moodle.

Les équipes

Le projet sera réalisé en **équipe de strictement 4 ou 5 étudiants**. Il y aura 8 équipes maximum par classe. Par exemple, dans une classe de 35 étudiant-e-s, il y aura strictement 3 équipes de 5 étudiant-e-s et 5 équipes de 4 étudiant-e-s.

Votre délégué-e récupérera la liste des équipes et la transfèrera à votre enseignant-e.

Le rapport

Un rapport de 4 pages maximum (en excluant les annexes qui, elles, pourront être longues) est demandé pour chaque équipe pour synthétiser vos résultats et votre réflexion. S'il-vous-plaît, utilisez $\text{\LaTeX}$.

La soutenance

Chaque équipe présentera son travail lors d'une soutenance orale de 10 minutes maximum, suivie de 40 minutes de questions. Vous présenterez les résultats de votre premier dépôt.

Pour l'oral, il est attendu une présentation avec des diapositives. La pédagogie et la clarté sont le but de cette présentation. Les diapositives ne doivent évidemment pas comporter de texte écrit à la main et photographié. Toutes les formules et matrices doivent être tapées.

A l'oral, il est demandé de restituer avec précision les points les plus importants de votre étude. Une diapositive qui résume le travail effectué est attendue. Attention, si vous dépassez les 10 min de présentation, votre enseignant-e vous arrêtera.

A l'issue de l'oral, des questions vous seront posées pendant 40 min : c'est un oral long où chaque étudiant sera interrogé individuellement sur le projet ou sur un point du cours. L'ensemble du projet devra être absolument maîtrisé par tous les membres de l'équipe. C'est principalement cet oral et la façon dont vous aurez tenu compte des conseils de votre enseignant.e pour votre dépôt final, qui comptera pour votre évaluation.

Le rendu

Le premier dépôt du projet de votre équipe se fera sur Moodle jusqu'au samedi 2 mai à 23h59. Aucun délai supplémentaire ne pourra être accepté, vos enseignant-e-s auront besoin du dimanche pour lire vos projets, sachant que les soutenances débutent pour certaines classes lundi à 8h.

Le dépôt final du projet de votre équipe se fera sur Moodle jusqu'au Samedi 13 juin à 23h59. Aucun délais supplémentaire ne pourra être accepté par vos enseignant-e-s.

Dans ces deux dépôts, vous joindrez votre rapport (premier dépôt, puis final), ainsi que vos annexes (programme, analyse préliminaire, etc). Le dossier de rendu de votre équipe sera intitulé de la façon suivante : pour le groupe I1_OPT-MATH1 et l'équipe 4 : "MML-MATH1-4". Et pour le groupe I1-INT_OPT-MATH1 et l'équipe 4 : "MML-INT-MATH1-4".

Bon courage pour la rédaction de ce projet,

L'équipe des enseignant-e-s de *Mathématiques pour le Machine Learning*.